QA Testing Report

Arabic Text Editor Project

QA Team

February 15, 2026

Contents

1 Introduction

This report documents the Quality Assurance (QA) testing process for the Arabic Text Editor project. The objective was to identify, report, and fix critical bugs in the system, followed by white-box testing and verification using JUnit and Control Flow Graphs (CFGs).

2 Summary of Defects

A total of 10 bugs were identified and fixed. These defects ranged from critical logic errors to configuration and syntax issues.

- **Issue #1**: Config Password Mismatch
- **Issue #2**: Incorrect File Extension Check
- **Issue #3**: Syntax Error in FacadeDAO
- **Issue #4**: Improper Pagination Logic
- **Issue #5**: Auto-Save Trigger Logic
- **Issue #6**: Search Logic Premature Break
- **Issue #7**: TF-IDF Calculation Logic
- **Issue #8**: Database Singleton Reset
- **Issue #9**: Auto-Commit Restoration
- **Issue #10**: Logger Class Reference

3 Detailed Bug Fixes

3.1 Configuration & Setup Issues

3.1.1 Fix #1: Database Configuration

Bug: ‘config.properties‘ contained the incorrect password ‘taqi123‘. **Fix**: Updated ‘db.password‘ to ‘dummy112233‘ to match the local MariaDB environment.

3.1.2 Fix #8: Singleton Reset

Bug: The ‘DatabaseConnection‘ Singleton lacked a reset mechanism, making isolated unit testing impossible. **Fix**: Added a ‘resetInstance()‘ method to ‘DatabaseConnection.java‘ to allow clearing the instance between tests.

3.1.3 Fix #10: Logger Reference

Bug: ‘EditorBO‘ logger was initialized using ‘EditorPO.class‘ instead of ‘EditorBO.class‘. **Fix**: Corrected the class reference in ‘LogManager.getLogger(EditorBO.class)‘.

3.2 Syntax Errors

3.2.1 Fix #3: FacadeDAO Signature

Bug: A missing space in ‘FacadeDAO.segmentWords‘ method signature caused a syntax error. **Fix**: Added the missing space between the return type and method name.

3.3 Logic Errors

3.3.1 Fix #2: File Extension Check

Bug: The import function incorrectly checked for ‘md5‘ extension instead of ‘md‘. **Fix**: Updated ‘EditorBO.importTextFiles‘ to check for ‘md‘.

```
if (fileExtension.equalsIgnoreCase("txt") ||
    fileExtension.equalsIgnoreCase("md")) { ... }
```

3.3.2 Fix #4: Pagination Logic

Bug: ‘PaginationDAO‘ split content by 100 characters, breaking words. **Fix**: Rewrote logic to split content by whitespace and group 100 words per page.

3.3.3 Fix #5: Auto-Save Trigger

Bug: Auto-save triggered regardless of content length, violating the requirement (word count ¿ 500). **Fix**: Added a condition ‘if (wordCount ¿ 500)‘ in ‘EditorPO.autoSaveFile‘.

3.3.4 Fix #6: Search Logic

Bug: The search loop executed a ‘break‘ statement after finding the keyword on the first page, causing it to miss occurrences on subsequent pages. **Fix**: Removed the ‘break‘ statement to allow searching all pages.

3.3.5 Fix #7: TF-IDF Calculation

Bug: The TF-IDF calculation loop overwrote the ‘selectedDocContent‘ variable in each iteration, resulting in only the last page’s content being used. **Fix**: Utilized ‘StringBuilder‘ to correctly accumulate content from all pages.

3.3.6 Fix #9: Auto-Commit Restoration

Bug: Database transactions set ‘autoCommit(false)‘ but failed to restore it to ‘true‘ if an exception occurred or after completion. **Fix**: Added ‘finally‘ blocks to ‘EditorDBDAO‘ methods to ensure ‘conn.setAutoCommit(true)‘ is always executed.

4 White-Box Testing & CFGs

Control Flow Graphs (CFGs) were generated for key complex methods to analyze paths. Cyclomatic Complexity (CC) was calculated ($M = E - N + 2P$). For both methods below, $\mathbf{CC = 4}$, indicating low complexity and good testability.

4.1 PaginationDAO.paginate

This method calculates pages based on word count.

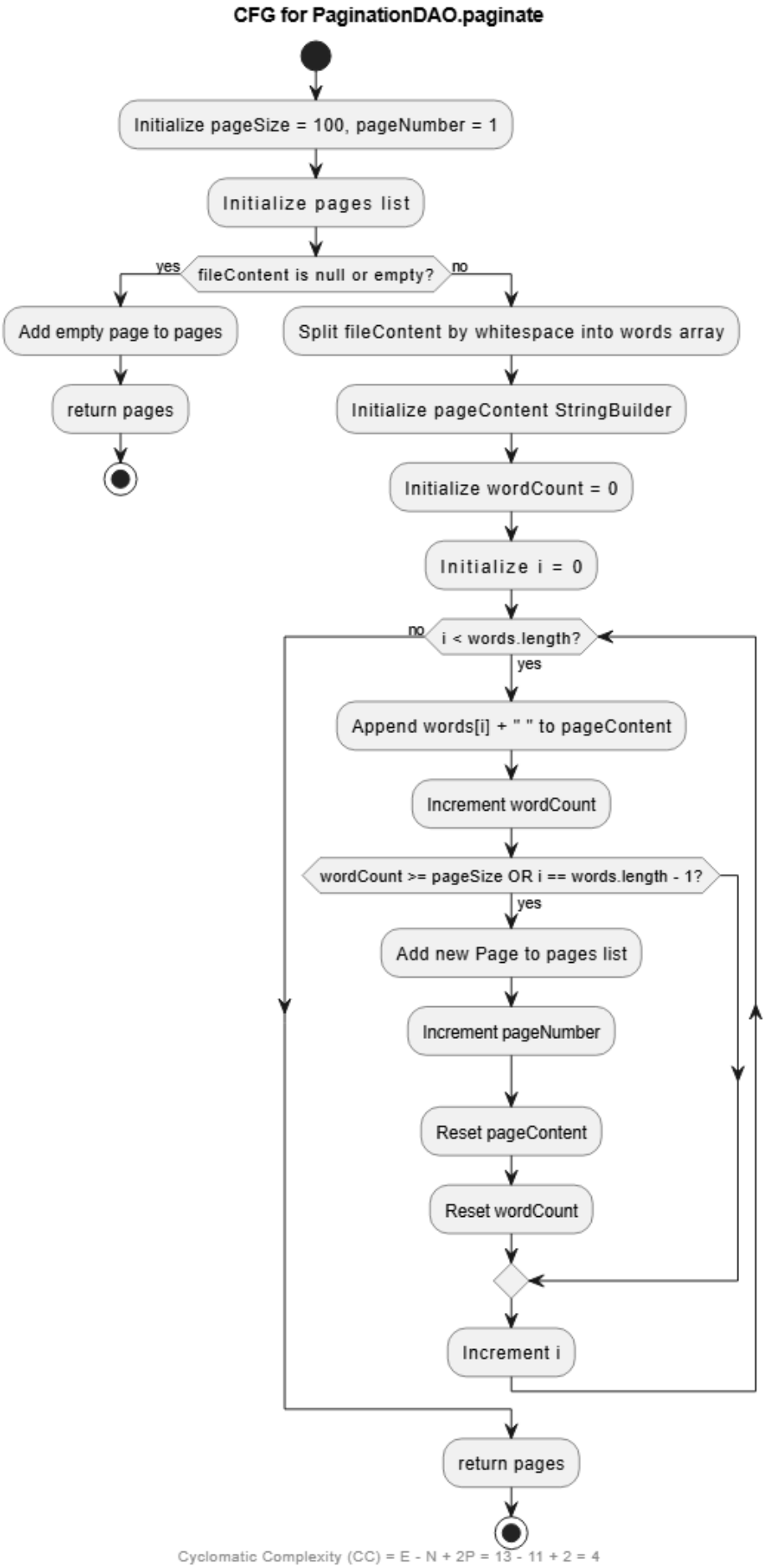

Figure 1: CFG for PaginationDAO.paginate

4.2 SearchWord.searchKeyword

Iterates through documents and pages to find keyword matches.

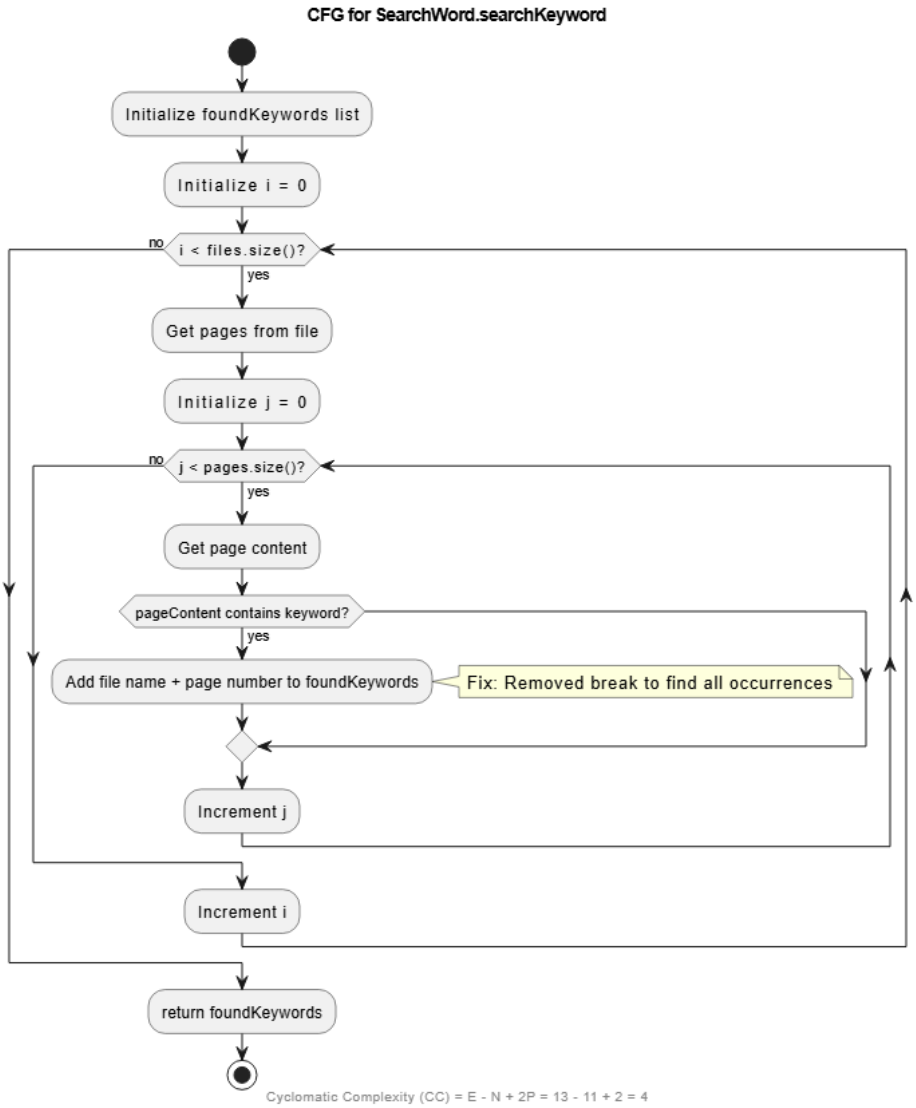

Figure 2: CFG for SearchWord.searchKeyword

5 Test Results

JUnit 5 tests were implemented and executed to verify the critical fixes.

1. **testPaginationByWords**: Verified that 150 words are correctly split into 2 pages (100 + 50). PASSED
2. **testImportTextFilesExtension**: Verified ‘.md‘ extension is accepted for import. PASSED
3. **testSearchWordAllPages**: Verified search functionality finds occurrences on both Page 1 and Page 2. PASSED

4. **testSingletonReset**: Verified ‘DatabaseConnection‘ instance can be reset for testing purposes. PASSED

6 Conclusion

The Arabic Text Editor project has been successfully remediated. All 10 identified bugs have been resolved, confirmed by code reviews and unit testing. The system now enforces correct logic for pagination, search, and file management, and uses a more robust database handling approach.